One Size Does Fit All: Exploring Model Fusion for Software Engineering Tasks

This document aims to provide instructions for using the artifact of the paper “One Size Does Fit All: Exploring Model Fusion for Software Engineering Tasks” (FSE26 Research paper). The repository for this study has been publicly archived on Zenodo (https://doi.org/10.5281/zenodo.19491719), containing the source code, datasets, and fine-tuned model checkpoints. You can unzip the ModelFusion.zip to get the repository and by using it, you can reproduce the main experimental results and figures presented in the paper (RQ1-RQ4). Please note that all commands in this document should be executed in the project root directory.

1. Artifact Structure

```
ModelFusion/
├── Analyse/                    # All figures in the paper
├── checkpoints/                # Checkpoints used in the paper
│   ├── GenerativeTask/         # Checkpoints for program generation tasks
│   ├── UnderstandingTasks/     # Checkpoints for program understanding tasks
│   └── learnable_fusion_weights/ # Checkpoints for learnable fusion parameters fusion
│       ├── MPLF/               # Checkpoints for multi-programming-language fusion
│       └── MSETF/              # Checkpoints for multi-SE-task fusion
├── data                        # Datasets used in the paper
├── SoftwareFusion              # Scripts for fusion
│   ├── MPLF/                   # Scripts for multi-programming-language fusion
│   │   ├── configs/            # Configuration files for scripts
│   │   ├── artifacts/          # Fusion results of different fusion approach
│   │   ├── code_repair_fusion.py                                # Multi-programming-language fusion for code repair
│   │   ├── code_repair_fusion_with_scaling_and_mask.py          # Multi-programming-language fusion for code repair with masks
│   │   ├── code_search_fusion.py                                # Multi-programming-language fusion for code search
│   │   └── code_search_fusion_with_scaling_and_mask.py          # Multi-programming-language fusion for code search with masks
│   └── MSETF/                  # Scripts for multi-software-engineering-task fusion
│       ├── configs/            # Configuration files for scripts
│       ├── artifacts/          # Fusion results of different fusion approach
│       ├── fusion_all.py                                       # Multi-SE-tasks and multi-programming-languages fusion
│       ├── fusion_all_with_scaling_and_mask.py                 # Multi-SE-tasks and multi-programming-languages fusion with masks
│       ├── program_generation_fusion.py                        # Program generation tasks fusion for python
│       ├── program_generation_fusion_with_scaling_and_mask.py  # Program generation tasks fusion with masks for python
│       ├── program_understanding_fusion.py                     # Program understanding tasks fusion for java
│       └── program_understanding_fusion_with_scaling_and_mask.py  # Program understanding tasks fusion with masks for java
├── CloneDetection/             # Fine-tuning scripts for clone detection task
├── CodeSearch/                 # Fine-tuning scripts for code search task
├── GenerativeTask/             # Fine-tuning scripts for program generation tasks
└── VulnerabilityDetection      # Fine-tuning scripts for vulnerability detection task
```

2. System Requirements

To ensure successful reproduction, the environment should meet the following requirements:

Hardware: A GPU with at least 32 GB of VRAM (tested on Nvidia 5090).

System Resources: At least 128 GB of system memory (RAM).

Software Environment: Operating System: Any 64-bit Linux distribution (recommended: AlmaLinux 9.5, which has been tested); Anaconda installed (for python environment management); Docker installed (for containerized execution); CUDA 12.8 or later.

For the Python environment, you can run the following command to install it.

```
conda create -n SEFusion python=3.10
conda activate SEFusion
pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cu128
pip install -r requirements.txt
```

For the evaluation environment of the code repair task, use the following command to create and run a Docker container, which will provide the evaluation service locally on port 5000.

```
cd GenerativeTask/ExecEval
docker build -t exec-eval:1.0 .
cd ../../
```

3. Quick Start

After all running requirements are satisfied, you can assess the performance of Scaling-Masks (RQ4) by using the following instructions.

```
cd SoftwareFusion
bash eval_scaling_masks.sh
cd ..
```

The above command will sequentially evaluate the performance of Scaling-Masks under the following fusion settings:

- Multi-programming-language fusion for code search tasks, including python, java, javascript, go, php, ruby.
- Multi-programming-language fusion for code repair tasks, including python, java, javascript, go, php, ruby.
- The fusion of all six tasks, including three Java program understanding tasks (code search, clone detection, and vulnerability detection) as well as three Python program generation tasks (code generation, code summarization, and code repair).

For more detailed information for other fusion approaches, please refer to the SoftwareFusion/README.md file in the ModelFusion.zip in the online repository.

4. Artifact Status

We would like to apply for the Artifacts Evaluated – Functional and Artifacts Available badges for the following reasons.

Artifacts Evaluated – Functional. Our artifact includes the full source code, the datasets used in the study, the fine-tuned checkpoints, and detailed instructions for installation and execution. These materials are directly relevant to the paper and support the generation of its main experimental results. We also provide the necessary scripts and documentation so that reviewers can run the artifact and reproduce the reported outputs in the paper. Therefore, we believe the artifact satisfies the criteria of being documented, consistent, complete, and exercisable.

Artifacts Available. Our research repository has been archived and made publicly accessible through Zenodo, which provides a persistent DOI and long-term availability. This ensures that the artifact can be permanently retrieved by reviewers and future researchers. Since the archived repository contains author-created materials that are relevant to the paper and adds substantial value beyond the text of the manuscript, we believe it satisfies the requirements for the Artifacts Available badge.